

Case 3: Joint Space Interpolation Motion Planning

MoveIt Task Constructor on the xArm 7, E-Waste Battery Extraction, Redback Operations

Prepared by Tarunjeet Singh, Project Lead

1. Purpose

This document records the third motion planning variant. Every stage is solved by the MTC JointInterpolationPlanner, which solves inverse kinematics for the goal and then moves all seven joints linearly and synchronously from the start angles to the goal angles [1], [2]. The scene, task stages and execution settings are identical to Cases 1 and 2.

2. Environment

Component	Version or detail
Host	Windows laptop running WSL2
Operating system	Ubuntu 24.04 LTS (Noble) in WSL2
Middleware	ROS 2 Jazzy Jalisco
Motion planning	MoveIt 2 (Jazzy binary packages)
Task planning	MoveIt Task Constructor (Jazzy binary packages)
Robot package	xarm ros2, jazzy branch, built from source in ~/xarm_ws
Simulation	xarm7_moveit_fake.launch.py with ros2_control fake hardware
Project package	xarm7_mtc_planners, node mtc_planner_compare (C++)

3. Scene and Task Structure

Item	Definition
Battery	Box 70 x 50 x 10 mm at (0.40, 0.00, 0.005) m in link_base, shown in orange
Drop zone	Pad 120 x 120 x 5 mm at (0.30, -0.30, 0.0025) m, shown in green
Planning group	xarm7 with IK frame link_eef, tool pointing down for all pose goals
Execution speed	Velocity and acceleration scaling factor 0.15
Hold between stages	2.0 s pause after every motion and every scene event

#	Stage	MTC stage type	Motion
0	Current state	CurrentState	Reads the live robot state and scene
1	Move above battery	MoveTo (pose)	To (0.40, 0.00, 0.20) m
2	Descend to battery	MoveRelative	0.15 m down in z
3	Vacuum ON	ModifyPlanningScene	Attach battery to link_eef
4	Lift battery	MoveRelative	0.15 m up in z
5	Transfer to drop zone	MoveTo (pose)	To (0.30, -0.30, 0.20) m

#	Stage	MTC stage type	Motion
6	Lower battery	MoveRelative	0.12 m down in z
7	Vacuum OFF	ModifyPlanningScene	Detach battery
8	Retreat	MoveRelative	0.12 m up in z
9	Return home	MoveTo (joints)	All seven joints to 0 rad

4. Planner Configuration

No search or tool path constraint is applied. The interpolated path is checked for collisions, but the solver cannot route around an obstacle; if the direct joint path collides, the stage fails [3].

```

} else if (type == "joint") {
    // Case 3: linear interpolation in joint space, all joints move in sync
    s = std::make_shared<mtc::solvers::JointInterpolationPlanner>();
}

```

5. How to Run

Terminal 1: start the simulated xArm 7 with MoveIt and RViz.

```

source /opt/ros/jazzy/setup.bash
source ~/xarm_ws/install/setup.bash
ros2 launch xarm_moveit_config xarm7_moveit_fake.launch.py

```

Terminal 2: build the package and run this case.

```

cd ~/xarm_ws
MAKEFLAGS="-j1" colcon build --packages-select xarm7_mtc_planners
source install/setup.bash
ros2 run xarm7_mtc_planners mtc_planner_compare --ros-args \
    --params-file ~/xarm_ws/xarm7_params.yaml -p planner:=joint

```

Optional parameters: speed (default 0.15), pause (default 2.0 s), countdown (default 5 s) and execute (default true). Every run appends one row to ~/mtc_results.csv.

The parameter file is prepared once, as described in the Case 1 document.

6. Results

Run	Outcome	Plan (ms)	Cost	Exec (s)	Joint path (rad)
1	Success	421.7	11.655	14.251	5.786

Metric definitions: planning time is the wall clock time of task.plan(), which includes reading the planning scene from move_group. Cost is the MTC solution cost from its default cost function, where lower is better. Execution time is the summed duration of all trajectories, excluding the demonstration pauses. Joint path length is the summed Euclidean distance between consecutive joint space waypoints.

7. Observations

- **Shortest motion:** the smallest joint path (5.79 rad) and execution time (14.25 s) of the three cases, since each stage takes the direct route in joint space.
- **Coordinated movement:** all joints start and stop together, which produces smooth and predictable arm motion.
- **Tool path not controlled:** the end effector follows a curved path between poses, so vertical approach and lift moves are not guaranteed to stay vertical.
- **Planning time:** 421.7 ms on a single run, between the other two cases. Most of this time is fixed overhead such as reading the planning scene, so it does not reflect the low computational cost of interpolation itself.
- **Limitation:** no obstacle avoidance, so it is only suitable for motions through space known to be clear, such as returning home.

8. References

- [1] MoveIt Contributors, "Pick and place with MoveIt Task Constructor," MoveIt 2 Documentation. [Online]. Available: https://moveit.picknik.ai/main/doc/tutorials/pick_and_place_with_moveit_task_constructor/pick_and_place_with_moveit_task_constructor.html. Accessed: Sep. 28, 2026.
- [2] M. Görner, R. Haschke, H. Ritter, and J. Zhang, "MoveIt! Task Constructor for task level motion planning," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), Montreal, Canada, 2019, pp. 190–196.
- [3] Open Robotics, "moveit_task_constructor API documentation (ROS 2 Jazzy)." [Online]. Available: https://docs.ros.org/en/ros2_packages/jazzy/api/moveit_task_constructor_demo/. Accessed: Sep. 28, 2026.
- [4] UFACTORY, "xarm_ros2: ROS 2 developer packages for robotic products from UFACTORY," GitHub, jazzy branch. [Online]. Available: https://github.com/xArm-Developer/xarm_ros2. Accessed: Sep. 28, 2026.
- [5] MoveIt Contributors, "MoveIt 2 Documentation." [Online]. Available: <https://moveit.picknik.ai/main/index.html>. Accessed: Sep. 28, 2026.
- [6] ros2_control Contributors, "ros2_control Documentation (Jazzy)." [Online]. Available: <https://control.ros.org/jazzy/index.html>. Accessed: Sep. 28, 2026.